Project - High Level Design

on

Dockerized AI Marketing Planning Service

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Kavish Jha	EN22CS301494
2.	Kritika Purohit	EN22CS301527
3.	Kunal Patidar	EN22CS301538
4.	Jatin Patidar	EN22CS301458
5.	Javesh Khargonkar	EN22CS301460
6.	Krishna Pawar	EN22CS301524

Group Name: Group 06D5

Project Number: DO-06

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 2025 - 26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **Dockerized AI Marketing Planning Service** is a web-based intelligent application designed to generate structured marketing strategies from user-defined goals. The system combines an AI-driven planning approach with containerization and deployment practices to deliver a portable, consistent, and extensible solution.

The application allows users to enter a marketing objective, process it through an AI-powered planning agent, and receive a detailed marketing plan. In addition, the generated plans are stored in a MySQL database for retrieval, search, deletion, and export. The system is developed using **Python, Streamlit, LangChain, and MySQL**, and is packaged using **Docker** for environment consistency. It also incorporates **GitHub Actions** for CI and **Kubernetes manifests** for orchestration readiness.

1.1 Scope of the document

This document describes the high-level architecture and design of the Dockerized AI Marketing Planning Service. It covers application structure, process flow, data design, interfaces, session handling, deployment view, security considerations, and non-functional aspects.

1.2 Intended Audience

This document is intended for:

- faculty evaluators
- project mentors
- developers and maintainers
- students and reviewers
- DevOps and deployment stakeholders

1.3 System overview

The system provides an interactive interface where users can:

- enter a marketing goal
- choose response style
- generate a structured marketing plan
- export the plan as PDF
- store and manage plan history in MySQL

The application follows a simple layered design:

- frontend interface using Streamlit
- agent orchestration layer using LangChain
- helper tools for budget, keyword, ad, and schedule support
- persistent data storage using MySQL

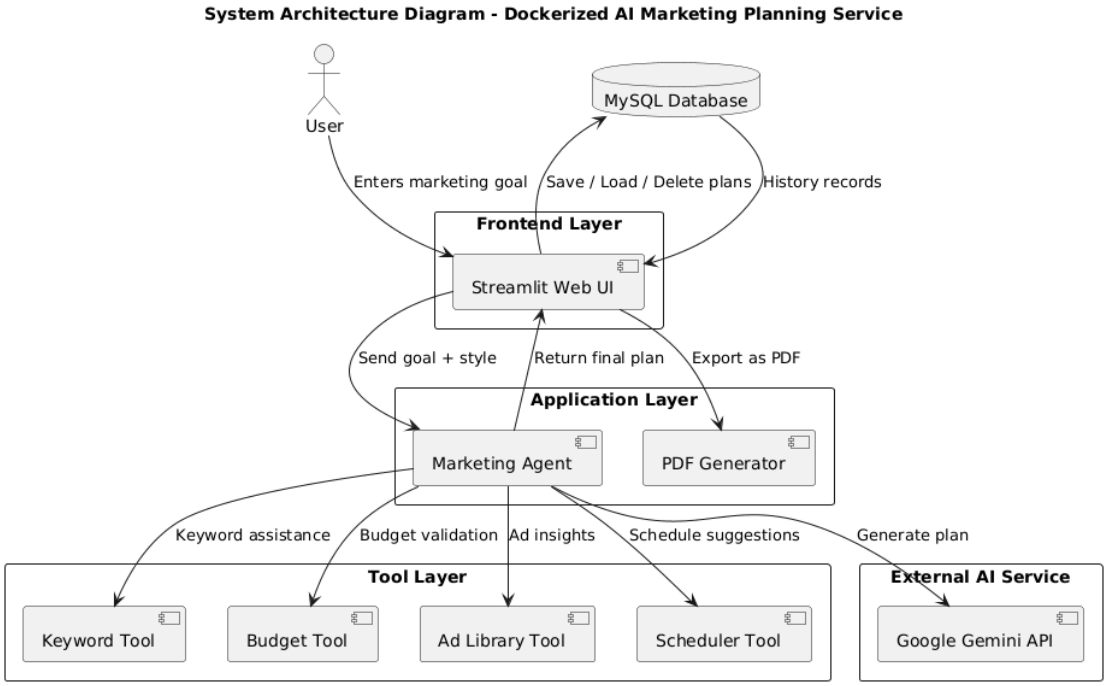
- containerized deployment using Docker and Docker Compose
- optional orchestration via Kubernetes

2. System Design

2.1 Application Design

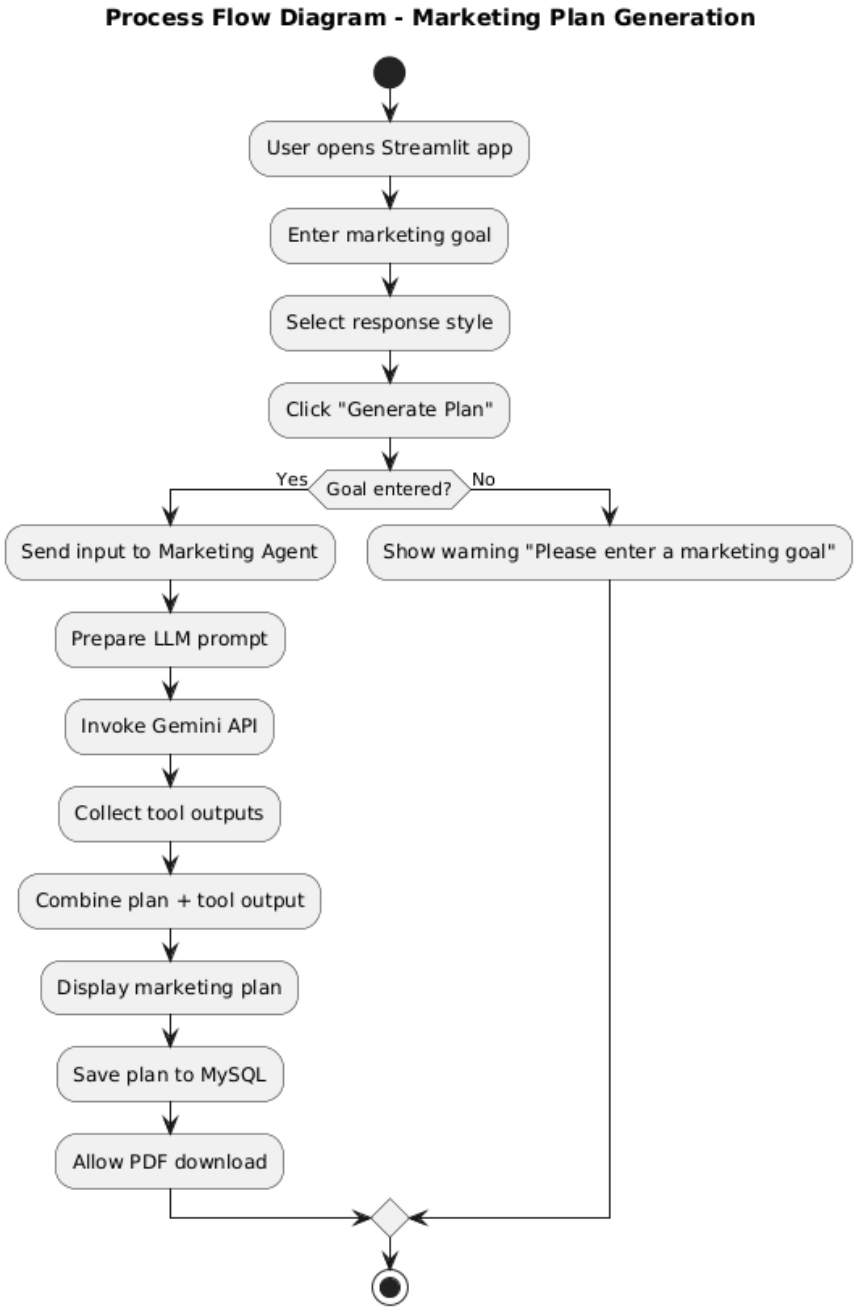
The system is composed of the following main modules:

1. **User Interface Layer**
   Built using Streamlit. It provides forms, plan generation controls, history panel, PDF download, and display components.
2. **Agent Layer**
   Implemented in agent.py. It receives user goals, sends prompts to the LLM, and composes the final marketing output.
3. **Tool Layer**
   Includes utility modules such as:
   - ad library tool
   - keyword tool
   - budget tool
   - scheduler tool
4. **Data Access Layer**
   Implemented in database_mysql.py. It handles save, load, delete, and clear-history operations.
5. **Deployment Layer**
   Includes Dockerfile, Docker Compose, GitHub Actions CI pipeline, and Kubernetes manifests.



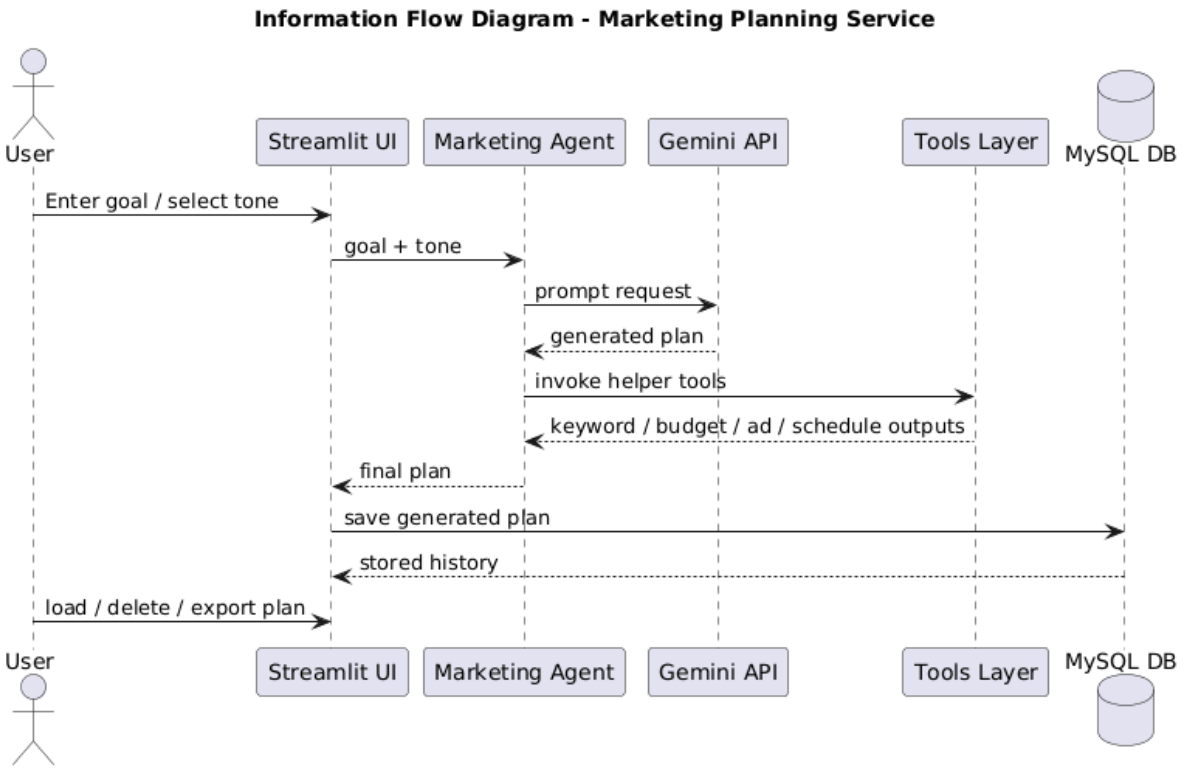
2.2 Process Flow

1. User opens the application.
2. User enters a marketing goal and selects response style.
3. Streamlit sends the goal to the agent layer.
4. Agent prepares prompt and invokes LLM.
5. Tool outputs are optionally combined into final result.
6. Generated plan is displayed on screen.
7. Plan is stored in MySQL database.
8. User can reload, search, delete, or export stored plans.



2.3 Information Flow

- User input flows from Streamlit UI to the Marketing Agent.
- Agent sends structured prompt to LLM.
- Agent combines LLM response with tool-based output.
- Final plan is returned to UI.
- Plan is written to MySQL for persistence.
- On future access, plan history is retrieved from MySQL and shown in sidebar.



2.4 Components Design

Major components include:

- **Streamlit Application (streamlit_app.py)**
  - renders UI
  - handles session state
  - loads history
  - displays generated and loaded plans
  - triggers PDF generation
- **Marketing Agent (agent.py)**
  - creates prompt
  - invokes Gemini model
  - integrates helper tool outputs
- **Configuration (config.py)**
  - reads environment variables

    - loads model and API settings
- **Database Layer (database_mysql.py)**
    - manages MySQL connectivity
    - CRUD operations for plans
- **Tools**
    - ad research helper
    - budget planning helper
    - keyword helper
    - schedule helper
- **Deployment Files**
    - Dockerfile
    - docker-compose.yml
    - init.sql
    - GitHub Actions workflow
    - Kubernetes YAML manifests

2.5 Key Design Considerations

- simple and modular architecture
- separation of UI, agent logic, and data access
- portability through Docker
- persistent history using MySQL
- support for local, Docker, and Kubernetes execution
- environment-based configuration
- basic CI validation using GitHub Actions
- ease of academic demonstration and future extensibility

2.6 API Catalogue

This project primarily uses internal function calls and external LLM API access rather than a custom REST API.

External API used:

- Google Generative AI / Gemini API for text generation

Internal callable operations:

- save_plan(goal, plan)
- load_plans()
- delete_plan(plan_id)
- clear_history()
- generate_pdf(text)
- MarketingAgent.run(goal)

3. Data Design

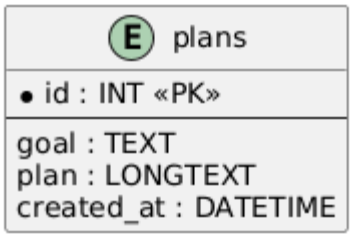
3.1 Data Model

The core database table is plans.

Table: plans

- id – integer, primary key, auto increment
- goal – text, stores user marketing goal
- plan – long text, stores generated marketing plan
- created_at – datetime, stores generation timestamp

Data Model Diagram - plans table



3.2 Data Access Mechanism

The application uses Python MySQL Connector to connect with MySQL.
Database operations are encapsulated in database_mysql.py.
The app performs:

- insert on plan generation
- select for history loading
- delete for individual record deletion
- delete all for history clearing

3.3 Data Retention Policies

Currently, generated plans are retained in the MySQL database until:

- user deletes a single plan
- user clears all history
- database is reset manually

No automated archival or retention expiration policy is implemented in the current version.

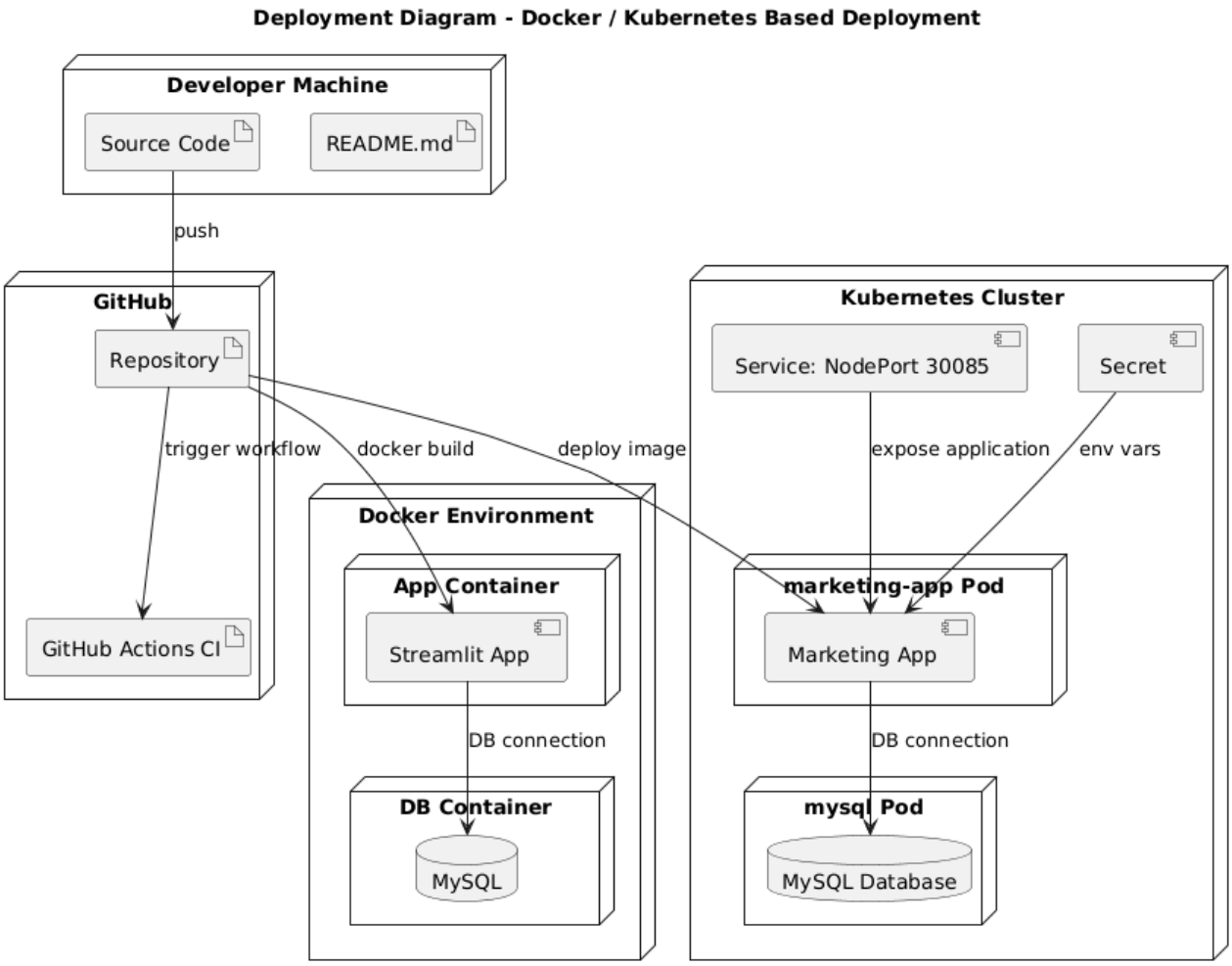
3.4 Data Migration

At present, no formal migration framework is implemented.
Database initialization is handled using init.sql in Docker Compose based deployment.
In Kubernetes, the table may require manual initialization or future automation through ConfigMap/init container.

4. Interfaces

The system provides the following interfaces:

1. **User Interface**
   - Streamlit-based web interface
   - accessible through browser
   - local URL or container-exposed port
2. **Database Interface**
   - MySQL connection using Python connector
   - SQL operations for persistence
3. **LLM Interface**
   - Google Gemini API through LangChain wrapper
4. **Deployment Interface**
   - Docker CLI
   - Docker Compose
   - kubectl for Kubernetes deployment
   - GitHub Actions workflow execution



5. State and Session Management

The application uses Streamlit session state for temporary UI-level state management.

Key session variables:

- `history` – stores loaded plan records during session
- `selected_plan` – stores currently viewed historical plan

Session state helps preserve currently loaded plan and history list during user interaction without requiring page redesign.

Persistent storage is handled separately in MySQL.

6. Caching

No dedicated caching layer such as Redis is implemented in the current system.

Current optimization approach includes:

- loading plan history into session state
- refreshing history only when needed
- avoiding repeated unnecessary reloads during UI interaction

Future scope may include:

- Redis-based caching
- LLM response caching
- query result caching

7. Non-Functional Requirements

7.1 Security Aspects

- API keys and database credentials are managed through environment variables
- `.env` is excluded from public source control
- Kubernetes secrets are used for deployment configuration
- database operations are isolated through utility methods
- Dockerized execution provides environment consistency
- future scope may include authentication, role-based access, and secret managers

7.2 Performance Aspects

- Docker ensures consistent runtime environment
- MySQL provides persistent structured storage
- session state reduces redundant reloading
- GitHub Actions validates builds automatically

- Kubernetes manifests allow scalable future deployment
- application performance depends on:
  - API response time from Gemini
  - local/container resource allocation
  - MySQL availability
  - Docker/Kubernetes startup overhead

8. References

- Python Documentation
- Streamlit Documentation
- LangChain Documentation
- MySQL Documentation
- Docker Documentation
- Docker Compose Documentation
- GitHub Actions Documentation
- Kubernetes Documentation
- Google Generative AI / Gemini API Documentation